

JamScript

JamScript é inspirada na sintaxe do JavaScript tradicional, porém com escopo estático, conjunto reduzido de construções e sem suporte a objetos, protótipos ou funções aninhadas. A linguagem suporta:

- Variáveis declaradas com `let` e constantes com `const`
- Funções com parâmetros e retorno
- Tipagem estática simples
- Operadores aritméticos, relacionais e lógicos
- Comandos de controle (`if`, `while`, `for`, `break`)
- Funções nativas de entrada e saída: `print()` e `input()`

O JamSript deve ser processado por um compilador com análise léxica, sintática e semântica, sendo proibido aceitar qualquer construção não definida nesta especificação.

1. Tipos básicos e derivados

JamScript fortemente tipada, diferenciando-se da sua inspiração JavaScript, e possui os seguintes tipos básicos:

- `int` — números inteiros
- `float` — números reais
- `bool` — valores booleanos (`true` ou `false`)
- `string` — textos

Não há arrays, objetos, funções anônimas ou funções como valores.

Tipos derivados são definidos com:

```
struct <tipo> {  
    <declarações>  
}
```

E podem ser instanciados com:

```
let <id> : <tipo>;
```

E podem ser acessados com:

```
<id>.<atributo>
```

Exemplo:

```
struct Coordenada {  
    let x: float;  
    let y: float;  
}  
  
main {  
    let lugar : Coordenada;  
    lugar.x = 2.0;  
    lugar.y = 3.0;  
    print(lugar.x, ", ", lugar.y);  
}
```

2. Operadores e Identificadores

Apenas os operadores citados são considerados, de forma que exemplos como '===' da sua inspiração JavaScript não são aplicáveis no JamScript.

2.1. Operadores Unários

- '!': negação, inverte o valor booleano ao qual foi aplicada - unário;
- '-': menos unário, inverte o valor inteiro ou real ao qual foi aplicada - unário;

2.2. Operadores Aritméticos

- '+': soma – binário;
- '-': subtração – binário;
- '*': multiplicação – binário;
- '/': divisão – binário;

2.3. Operadores Relacionais

- '==': comparação, checa se os operadores são iguais - binário;
- '!=': comparação, checa se os operadores são diferentes - binário;
- '>': maior que - binário;
- '<': menor que - binário;

2.4. Operadores incrementais

- i++, i--, ++i, --i

Todos os operadores incrementam ou decrementam a variável antes de retornar ou utilizar seu valor na expressão. Não há diferença semântica entre formas pré-fixadas e pós-fixadas. Somente aplicáveis a variáveis dos tipos int, e nunca a constantes, literais ou expressões.

2.5. Precedência dos operadores

- I. Operadores de mesma precedência são avaliados da esquerda para a direita.
- II. Parênteses podem alterar a ordem natural.

A tabela a seguir apresenta a hierarquia de precedência dos operadores (decrecente de cima para baixo), os tipos dos operandos aos quais podem ser aplicados e o tipo do retorno:

Operador	Aridade	Tipos aplicados	Tipo de retorno
!	1	booleano	booleano
-	1	inteiro e real	inteiro e real
* e /	2	inteiro e real	inteiro e real
+ e -	2	inteiro e real	inteiro e real
++ e --	1	inteiro	inteiro
> e <	2	booleano, inteiro, real e string	booleano
== e !=	2	booleano, inteiro, real e string	booleano
&&	2	booleano	booleano
	2	booleano	booleano

Tabela 1. Precedência de operadores.

2.6. Identificadores

- Um identificador válido deve começar com letra ou _;
- Pode conter letras, dígitos ou _;
- Não pode ser palavra reservada (if, while, let, ...).
- Identificadores são usados para variáveis, constantes e funções.

3. Regras para uso de Variáveis e Constantes

Todas as variáveis e constantes devem ser declaradas antes do primeiro comando do bloco.

3.1 Sintaxe Geral

```
let <id> : <tipo>;  
let <id> : <tipo> = <expressão>;  
  
const <id> : <tipo> = <expressão>;
```

3.2 Tipos de Inicialização

- Variáveis podem ou não ter valor inicial.
- Constantes devem ser inicializadas obrigatoriamente.
- Tipos devem ser consistentes com a expressão atribuída.

3.3 Valores Padrão

Variáveis sem inicialização recebem:

Tipo	Valor padrão
int	0
float	0.0
bool	false
string	""

Exemplos

```
let x : int;  
let y : float = 3.14;  
const PI : float = 3.1415;  
let nome : string = "Maria";
```

3.4 Escopo das variáveis e constantes

JamScript possui escopo local por bloco, semelhante ao JavaScript moderno.

Um novo escopo é criado para:

- Corpo de funções;
- Blocos { ... } de if, while, for.

É proibido redeclarar um identificador já presente no mesmo escopo.

4. Declaração de funções

Funções são definidas com:

```
function <nome> ( <lista_parâmetros> ) : <tipo_retorno> {  
    <declarações>  
    <comandos>  
    return <expressão>;  
}
```

- Funções não podem ter o mesmo nome que outra função, variável ou constante.
- Parâmetros são passados por valor.
- Não há funções aninhadas.
- Função sem retorno explícito deve ser declarada como : void.

Exemplo

```
function soma(a : int, b : int) : int {
    let r : int;
    r = a + b;
    return r;
}
```

5. Funções nativas

5.1. print()

Exibe valores no console.

Sintaxe:

`print(expr1, expr2, ...);`

Exemplo: `print("Resultado: ", x);`

5.2. input()

Lê inteiros, floats e strings do teclado.

Sintaxe:

`input(x);`

Exemplo: `input(a, b);`

6. Sentenças de atribuição e controle

6.1 If-Else

JamScript suporta apenas as formas if e if-else; Não existe uma construção como else if, ou variações análogas de condicional de natureza ternária ou múltipla escolha.

```
if (condicao) {
    ...
} else {
    ...
}
```

Exemplo:

```
let x : int = 10;

if (x > 5) {
    print("maior que 5");
} else {
    print("menor ou igual a 5");
}
```

6.2 While

```
while (condicao) {
    ...
    [break;]
}
```

Exemplo:

```
let i : int = 0;

while (i < 3) {
    print(i);
    i++;
}
```

6.3 For

```
for (inicialização; condição; incremento) {  
    ...  
    [break;]  
}
```

exemplo:

```
for (i = 1; i <= 3; i++) {  
    print("valor: ", i);  
}
```

Incremento pode ser `i++`, `i--` ou uma atribuição simples

7. Método main

Um programa JavaScript é composto pela palavra-chave **program**, seguida de um nome e ponto-e-vírgula, depois funções e finalmente o método principal **main**.

Estrutura Geral

```
program MeuPrograma;  
  
function f(x : int) : int {  
    ...  
}  
  
main {  
    <declarações>  
    <comandos>  
}
```

- O bloco **main** não possui parâmetros.
- É obrigatório existir exatamente um **main**.

Exemplo Completo

```
program Exemplo;  
function dobro(a : int) : int {  
    let r : int;  
    r = a * 2;  
    return r;  
}  
  
main {  
    let x : int;  
    print("Digite um número:");  
    input(x);  
    print("Dobro: ", dobro(x));  
}
```

8. Erros Semânticos (que o compilador deve detectar)

- Uso de variável não declarada;
- Redecaração no mesmo escopo;
- Atribuição em constante;
- Tipos incompatíveis em expressões;
- Função sem *return* no caminho de execução;
- Chamada de função com número incorreto de argumentos;
- Comparações inválidas (ex.: `string > int`);
- Declarações após comandos dentro de um bloco.